Reviewer Pack — Minimal Quadratic Residues and Communication Complexity Rank…

Entry 2cf9f200237d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 10:04:05 UTC

Minimal Quadratic Residues and Communication Complexity Rank Variance

Entry ID: `2cf9f200237d` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-07 10:04:05 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (Quadratic Residue symbol) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

> For all Boolean functions f: {0,1}^n -> {0,1}, the variance of communication complexity rank V(f) is linearly correlated with the minimal quadratic residue degree D_min(f), such that V(f) = Θ(D_min(f)^2).

Rationale (proposer's reasoning):

> The connection between quadratic residues and number theory may provide insights into the structure of Boolean functions, which are central to communication complexity. If the communication complexity rank variance is proportional to the quadratic residue degree, it could suggest a fundamental relationship between these two areas.

Taxonomy category: `communication_complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `19ab7339dfc3d75b`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the correlation coefficient between V(f) and D_min(f)^2 over 30 random seeds is ≥ 0.8, with no seed producing a communication complexity rank > 10.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"quadratic residue" AND "communication complexity" AND "matrix rank"` - `"minimal quadratic residue" AND "variance of communication complexity" AND "rank variance"` - `"0 notation" AND "D_min(f)" AND "communication complexity"`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.4s

5.1 Generated Python source

```python
import random
import math
import sys
from fractions import Fraction

def generate_random_boolean_function(n):
    return [random.choice([0, 1]) for _ in range(2**n)]

def communication_complexity_rank(f):
    n = int(math.log2(len(f)))
    if len(f) != 2**n:
        raise ValueError("Function length must be a power of 2")

    min_bits = float('inf')
    for i in range(n):
        bits = [f[j] for j in range(2**n) if (j & (1 << i)) == 0]
        min_bits = min(min_bits, len(set(bits)))

    return math.ceil(math.log2(min_bits))

def minimal_quadratic_residue_degree(f):
    n = int(math.log2(len(f)))
    if len(f) != 2**n:
        raise ValueError("Function length must be a power of 2")

    residues = set()
    for x in range(1, 2**n):
        y = (x * x) % (2**n)
        residues.add(y)

    return len(residues)

def run_trial(seed: int) -> dict:
```

```
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    V_f_values = []
    D_min_f_squared_values = []
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        for _ in range(5):  # Test each n with 5 different functions
            f = generate_random_boolean_function(n)
            if communication_complexity_rank(f) > 10:
                continue

            V_f = communication_complexity_rank(f)
            D_min_f_squared = minimal_quadratic_residue_degree(f) ** 2

            V_f_values.append(V_f)
            D_min_f_squared_values.append(D_min_f_squared)
            instances_tested += 1

    if instances_tested < 30:
        return {
            "metric_name": "communication_complexity_rank_variance",
            "metric_value": None,
            "instances_tested": instances_tested,
            "n_max": max(n_values),
            "conjecture_holds": False,
            "counterexample": "insufficient_instances"
        }

    V_f_mean = sum(V_f_values) / len(V_f_values)
    D_min_f_squared_mean = sum(D_min_f_squared_values) / len(D_min_f_squared_values)
    correlation_coefficient = (sum((V_f - V_f_mean) * (D_min_f_squared - D_min_f_squared_mean) for
                               (len(V_f_values) * math.sqrt(sum((V_f - V_f_mean) ** 2 for V_f in V
                                math.sqrt(sum((D_min_f_squared - D_min_f_squared_mean) ** 2 for D_

    if correlation_coefficient < 0.8:
        conjecture_holds = False
        counterexample = f"correlation_coefficient={correlation_coefficient:.4f}"

    return {
        "metric_name": "communication_complexity_rank_variance",
        "metric_value": V_f_mean,
        "instances_tested": instances_tested,
        "n_max": max(n_values),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]]
    if not seeds:
        primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73,
        seeds = random.sample(primes, 30)
```

```python
results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {{\"seed\": {seed}, **{result}}}")
    results.append(result)

V_f_values = [r["metric_value"] for r in results if r["metric_value"] is not None]
support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={sum(V_f_values)/len(V_f_values):.4f} std={math.sqrt(sum((x
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={sum(V_f_values)/len(V_f_values):.4f} std={math.sqrt(sum((x
else:
    first_failing_seed = next(i for i, r in enumerate(results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means the correlation coefficient could not be calculated. | next: Re-run the test with increased time limits to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13969
2	preregistration	ollama_remote	glm4:latest	0	0	9184
3	novelty	ollama_remote	glm4:latest	0	0	8545
4	novelty	ollama_remote	glm4:latest	0	0	15409
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	47163
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10113
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15068
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15238
9	judge	ollama_remote	glm4:latest	0	0	26838

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 161527 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/2cf9f200237d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2cf9f200237d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/2cf9f200237d.tar.gz` (if generated)